

Lab 18 — Closed-Loop Control: PID, Anti-Windup and Derivative Kick

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2 hrs
Proves	steady-state offset, windup, anti-windup — measured
Prerequisites	Labs 2 and 4 (fixed-rate execution).
Beyond the board	Nothing — the plant is simulated on the chip.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§18.1 objective**, **§18.2 parts and wiring**, **§18.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure**, **Phase 1 the complete code**, walkthrough, **Phase 2 build and run**, **Phase 3 prove it**, then worked extensions. Every file is given in full, the type is named correctly, and nothing is left for you to work out.

18.1 Objective & TM4C bridge

Write an interview-grade discrete PID controller, run it against a simulated plant, then deliberately reproduce integral windup and fix it. PID is the most universally asked control topic in embedded interviews, and being able to write one from memory with the two practical refinements below puts you well ahead.

TM4C BRIDGE

Nothing bare-metal about this — it is discrete-time control mathematics. What connects it to bare-metal work is the timing discipline: a control loop **must** run at a fixed rate, which is the `vTaskDelayUntil` from Lab 2 and the reasoning from Lab 4, now with a concrete reason why jitter is not merely untidy but mathematically wrong.

18.2 Parts & wiring

Item	Status
ESP32-S3 board	nothing external — the plant is simulated in software

Simulating the plant is deliberate, not a compromise. A real motor adds friction, backlash, a noisy encoder and the risk of breaking something, all of which obscure the controller's behaviour while you are learning it. With a simulated first-order plant the response is repeatable, so when the output changes you know it was your gains and not the hardware. Once the controller is right, swapping in a real motor is a wiring problem rather than a control problem.

KEY POINT**Watch it as a graph**

The code prints CSV. Paste it into a serial plotter (the Arduino IDE has one, or use any spreadsheet) and you get the classic step-response curve. Tuning by staring at numbers is miserable; tuning by watching the curve is intuitive, and overshoot becomes something you can see rather than infer.

18.3 How it works

PID is the most universally asked control topic in embedded interviews, and it is genuinely useful. Assume no controls background; everything is below.

Open loop versus closed loop

You want a motor at 1000 rpm. **Open loop** means guessing the drive that ought to produce it — apply 60% power and hope. It fails the moment reality differs from your guess: the load changes, the battery sags, the bearing is cold, the motor ages. Nothing in the system notices or corrects.

Closed loop measures the actual output, computes the **error** (target minus measured), and continuously adjusts the drive based on that error. It does not need to know what power produces 1000 rpm; it discovers it, and re-discovers it whenever conditions change.

Two terms you will see constantly: the **setpoint** is where you want to be, the **process variable** (PV) is where you actually are. Error is setpoint minus PV, and the whole job is driving it to zero.

The three terms

Term	Responds to	Contributes	On its own it
P — proportional	Error <i>right now</i>	Immediate push, bigger when further away	Leaves a permanent offset

Term	Responds to	Contributes	On its own it
I — integral	Error <i>accumulated over time</i>	Grows until the offset is eliminated	Overshoots, and can wind up
D — derivative	How fast things are <i>changing</i>	Damping; anticipates overshoot	Amplifies noise badly

Why P alone always leaves an offset

This is a favourite interview question and the reasoning is short once you see it.

A proportional controller's output *is* the error multiplied by a gain: `out = kp × err`. Now suppose the error reached zero. Then the output would also be zero — but almost every real system needs a *nonzero* drive just to stay where it is. A motor needs power to overcome friction; a heater needs power to replace heat lost to the room.

So the system settles at whatever error produces exactly the drive needed to hold position. **The error cannot vanish, because the error is what is producing the drive.** Raising `kp` shrinks the offset but never removes it, and eventually makes the loop oscillate.

The integral term fixes this precisely because it keeps accumulating while *any* error remains. It supplies the steady drive, which lets the error go to zero, at which point the integral stops growing and holds its value.

Seeing all three at once

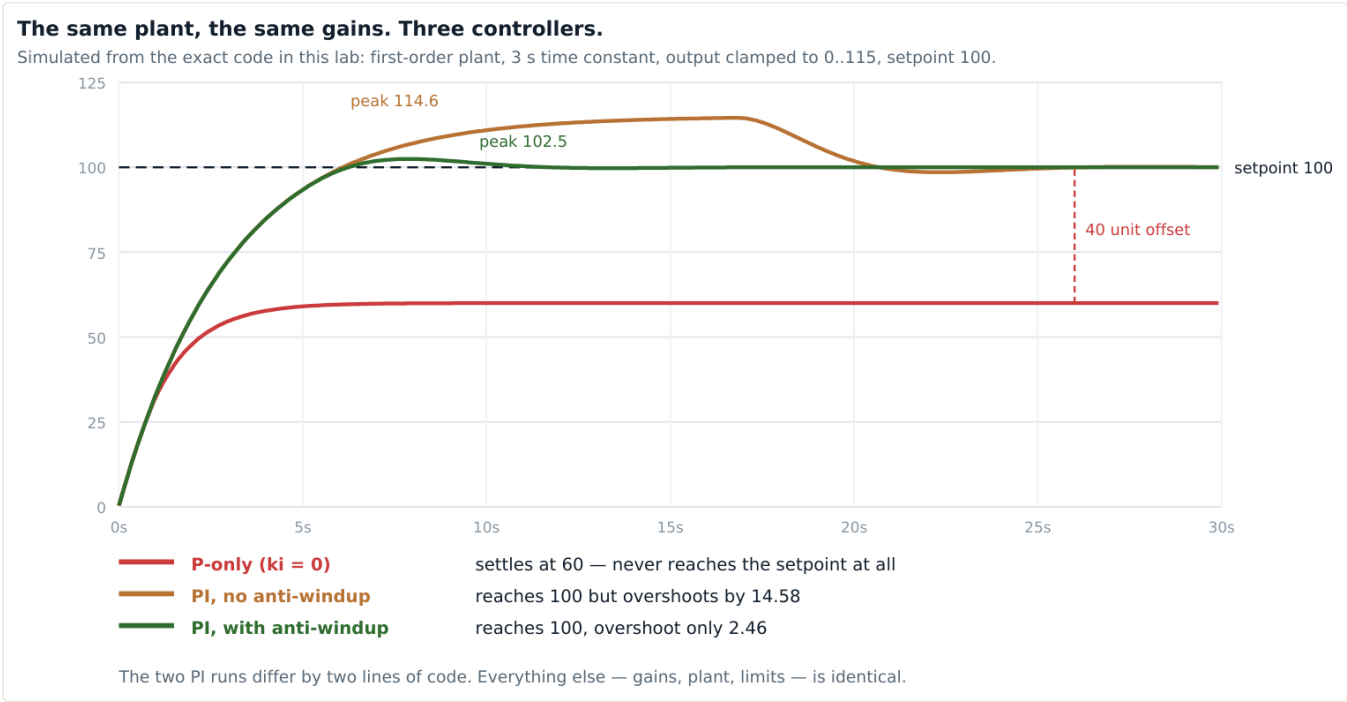


Figure 18.1. The three configurations you will run in §18.7, simulated from the exact code in this lab. Same plant, same gains, same output limits — the two PI curves differ by two lines of code. These are the numbers you should reproduce.

Simulated step response of three controllers, same plant and same gains.

Simulated from the exact code in this lab: first-order plant with a 3 second time constant, output clamped to 0..115, setpoint 100, loop at 100 Hz.

Controller	Result	Why
P-only (ki = 0)	settles at 60, never reaches the setpoint at all	steady-state offset of 40 units
PI, no anti-windup	reaches 100 but overshoots by 14.58, peaking at 114.6 and taking about 15 s to recover	integral wound up while the output was saturated
PI, with anti-windup	reaches 100, overshoot only 2.46, peaking at 102.5	conditional integration stops the integral growing while pinned

The two PI runs differ by two lines of code. Everything else -- gains, plant, limits -- is identical.

These are the numbers you should reproduce in section 18.7.

Read the red curve first: P-only climbs and then simply stops at 60, forty units short, for the reason above. It is stable and permanently wrong.

Both PI curves reach 100. The difference between them is what the rest of this section is about.

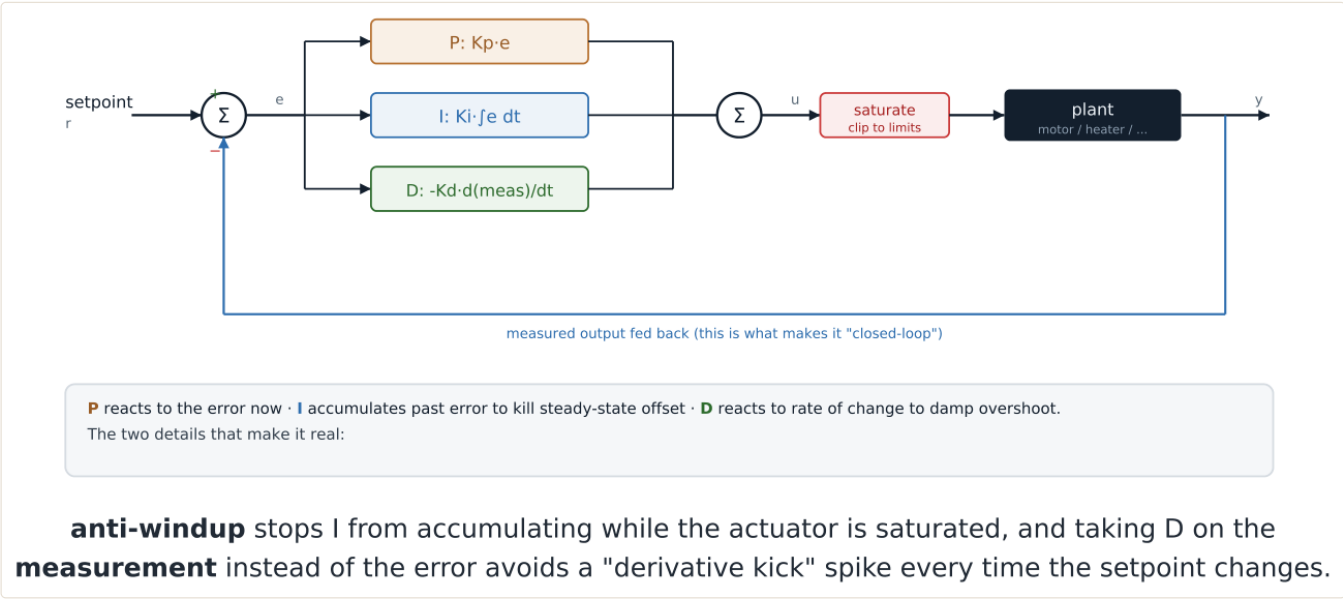


Figure 18.2. The loop as a block diagram. Error feeds three parallel terms whose sum drives the plant through a saturation block, and the measurement feeds back to the summing junction. Note the derivative is tapped from the *measurement*, not the error.

PID controller block diagram.

Signal path: setpoint -> summing junction (setpoint minus measurement) -> error **e** -> three PARALLEL terms -> second summing junction -> saturation block (clamp to output limits) -> plant (motor, heater, etc.) -> output.

The three parallel terms:

Term	Expression	Role
P	$K_p * e$	reacts to the error NOW
I	$K_i * \text{integral of } e \, dt$	accumulates past error to kill steady-state offset
D	$-K_d * d(\text{measurement})/dt$	reacts to rate of change, to damp overshoot

The measured output feeds back to the first summing junction. Note the derivative is taken from the MEASUREMENT, not from the error.

The two details that make it real: - Anti-windup stops the integral accumulating while the actuator is saturated. - Taking D on the measurement instead of the error avoids a derivative-kick spike every time the setpoint changes.

Derivative kick, and why D is taken on the measurement

The textbook form differentiates the *error*. That creates a nasty artefact: change the setpoint suddenly and the error jumps instantly, so its derivative is momentarily enormous, and the controller emits a violent output spike. In a real actuator that means a bang — a motor lurching, a valve slamming.

The fix is to differentiate the **measurement** instead. A physical quantity cannot jump instantly, so there is no spike. Since the setpoint is usually constant, the two forms are otherwise equivalent, and the sign flips as a consequence — which is why the code subtracts the D term rather than adding it.

This is a small change that marks out someone who has actually run a loop rather than only read the equation.

Integral windup: the bug anti-windup exists to prevent

Every real actuator saturates. A motor has a maximum speed; a heater has a maximum power; your code has `out_max`.

Now picture starting far from the setpoint. The output pins at its limit almost immediately — the system is going as fast as it physically can. But a large error persists for several seconds, and **the integral keeps accumulating the whole time**, even though extra integral does nothing at all: the output is already maxed and cannot go higher.

By the time the system reaches the setpoint, the integral has grown enormous. It now takes a long stretch of *negative* error — a large overshoot — to unwind all that accumulation. The controller looks badly tuned when in fact the arithmetic is wrong.

That is the orange curve in the figure: it sails past the setpoint to 114.6 and takes fifteen seconds to come back.



Figure 18.3. The mechanism in the time domain, computed from the model in §18.5. [SIMULATED] Read the middle panel: while the output is pinned the unprotected integrator climbs to 208 doing nothing useful, because the output physically cannot go higher. The protected one stops at 74. That excess is what must later be unwound, and unwinding it *is* the overshoot in the bottom panel.

Anti-windup in the time domain: output, integrator and process value. [SIMULATED] from the Lab 18 model. Three stacked panels, all sharing a 0-25 s time axis. Red = no anti-windup, green = conditional integration. Same gains, same plant, same limits.

A shaded band covers 0 to 16.8 s and is labelled: OUTPUT PINNED AT out_max (red curve).

Panel	Red (no anti-windup)	Green (conditional integration)
Controller output u (0-130)	sits flat on $out_max = 115$ from 0 to 16.8 s, then falls below and undershoots before settling	leaves the limit around 7 s and settles smoothly
Integrator $integ$ (0-230)	climbs to a peak of 208.49 , still 205 at $t = 5$ s	rises only to 73.58 , flat from about 7 s
Process value pv (0-130)	overshoots to 114.58	peaks at 102.46

The reading, in order:

- The top panel shows why the accumulation is wasted. While the output sits flat on `out_max`, every extra unit the integrator contributes changes the drive by exactly nothing. The integrator is writing cheques the actuator cannot cash.
- The middle panel is the mechanism. 208 versus 74 is the entire difference between the two runs.
- The bottom panel is the bill. The excess integral in red must later be unwound, and unwinding it IS the overshoot.

Note what the green curve does *not* do: it is not simply a smaller integral. It tracks the red one at the very start, then diverges the moment saturation begins, and resumes normal integration once the output comes off the limit around 7 s. **Anti-windup is not detuning** — the gains are identical. It only suppresses accumulation during the interval where accumulation is meaningless.

Anti-windup by conditional integration is the scheme this lab uses: while the output is saturated, commit the integration step *only* if the error has reversed and integrating would now pull the output back off the limit. The integral is frozen during the climb and resumes as soon as it can help. That is the green curve — overshoot drops from 14.58 to 2.46 with no change to the gains.

The other common scheme is **back-calculation**, which simply undoes the integration step whenever the output saturates, in either direction. §18.9.2 measures both: on this plant they are *identical at every sample*, for a reason worth reading. Be careful not to use the two names interchangeably — an earlier edition of this text did, and described the code as back-calculation when the listing plainly performs conditional integration.

KEY POINT

Why this is the detail worth remembering

Windup is not exotic. It happens on *any* loop with a limited actuator and a setpoint far from the current state, which is essentially every real system at startup. A PID implementation without anti-windup is not a simplified PID; it is a PID with a bug that hides until the first large step.

Why the loop must run at a fixed rate

Both the integral and the derivative multiply or divide by `dt`, and the code assumes `dt` equals the real elapsed time between calls. If the loop actually runs at irregular intervals, that assumption is false and the maths is *silently wrong* — the controller misbehaves in ways that look exactly like bad tuning.

A control loop must be periodic. That is the sentence to have ready, and it is why this lab uses the drift-free absolute-deadline timing rather than a plain delay.

How you would actually tune one

Worth knowing so you have an answer beyond "guess." A common practical method: set `ki` and `kd` to zero and raise `kp` until the output oscillates steadily, then back it off to roughly half. Add `ki` until the steady-state offset disappears, accepting some overshoot. Add `kd` only if you need to damp that overshoot and your measurement is clean enough to differentiate.

Ziegler-Nichols is the classic formalisation of that idea, and it is worth knowing by name. The honest answer to "how would you tune a loop" is that you start from a systematic method and then adjust against a measured step response — which is exactly what the figure above shows.

18.4 Phase 0 — configure

0.1 — sdkconfig.defaults

```
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_FREERTOS_HZ=1000
```

1 kHz tick so `vTaskDelayUntil` can hold a true 100 Hz loop. At the default 100 Hz tick, a 10 ms period is exactly one tick and any jitter is a whole period — which corrupts the `dt` the maths assumes.

0.2 — top-level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab18_pid)
```

0.3 — main/CMakeLists.txt

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS ".")
```

0.4 — apply and verify

```
Remove-Item sdkconfig -ErrorAction SilentlyContinue
idf.py fullclean
idf.py set-target esp32s3
idf.py reconfigure
Select-String -Path build\config\sdkconfig.h -Pattern "define CONFIG_FREERTOS_HZ"
```

KEY POINT

Expected

`#define CONFIG_FREERTOS_HZ 1000`. If it reads 100, `sdkconfig.defaults` was not picked up — it is only read when no `sdkconfig` exists, so delete `sdkconfig` and reconfigure again.

18.5 Phase 1 — the complete main.c

WARNING

Two corrections baked into this listing

The type is `pid_controller_t`, not `pid_t`. `pid_t` is a POSIX type declared in `<sys/types.h>` for process identifiers. Newlib pulls that header in transitively, so the original name can collide and produce a confusing redefinition error.

Anti-windup is conditional integration rather than backing the step out. See §18.9.2 for the measurement showing when that actually matters — the answer is more interesting than you would expect.

```
// =====
// Lab 18 -- PID
//
// Goal: Watch P-only leave a permanent offset, watch integral windup cause
// overshoot, then kill the windup without touching the gains.
// Verify: MODE 0 settles at 60.00 (offset 40.00). MODE 1 peaks 114.58.
// MODE 2 peaks 102.46. Printed by the summary block.
// Gotcha: The type is pid_controller_t. pid_t is POSIX (<sys/types.h>) and
// collides. dt is a CONSTANT in the maths, so the loop MUST be
// periodic -- vTaskDelayUntil, not vTaskDelay.
// =====
#include <stdio.h>
#include <stdbool.h>
#include <math.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_timer.h"
#include "esp_log.h"

// 0 = P-only          expect: settles at 60, never reaches 100
// 1 = PI, NO anti-windup expect: reaches 100, overshoots to ~114.6
// 2 = PI, anti-windup   expect: reaches 100, overshoots to ~102.5
#define MODE                2

#define DT_MS              10          // 100 Hz control loop
#define SETPOINT           100.0f
#define RUN_SECONDS        40
#define PLANT_TAU          3.0f        // first-order lag, 3 s time constant
#define OUT_MIN            0.0f
#define OUT_MAX            115.0f      // only just above the setpoint -- forces saturation

static const char *TAG = "lab18";

// ----- controller
typedef struct {
    float kp, ki, kd;
    float dt;
    float out_min, out_max;
    bool anti_windup;
    // --- state that must persist between calls ---
    float integ;
    float prev_meas;
    bool started;
} pid_controller_t;          // NOT pid_t -- that is POSIX, in <sys/types.h>

static float pid_step(pid_controller_t *p, float sp, float meas)
{
    float err = sp - meas;

    // Derivative on the MEASUREMENT, not the error. A setpoint change makes the
    // error jump instantly; differentiating that produces a huge output spike
    // ("derivative kick"). A physical measurement cannot jump, so no spike.
    // The sign flips as a consequence, which is why D is SUBTRACTED below.
    float deriv = p->started ? (meas - p->prev_meas) / p->dt : 0.0f;
    p->prev_meas = meas;
    p->started = true;
}
```

```

// Try the integration, but do not commit it yet.
float integ_trial = p->integ + err * p->dt;
float out = p->kp * err + p->ki * integ_trial - p->kd * deriv;

if (out > p->out_max) {
    out = p->out_max;
    // Commit only if this error pulls us back DOWN off the ceiling.
    if (!p->anti_windup || err < 0.0f) p->integ = integ_trial;
} else if (out < p->out_min) {
    out = p->out_min;
    if (!p->anti_windup || err > 0.0f) p->integ = integ_trial;
} else {
    p->integ = integ_trial;          // linear region: always integrate
}
return out;
}

// ----- plant
// First-order lag: moves toward the drive by a fraction of the remaining gap
// each step. Fast at first, slowing as it approaches -- how a motor's speed,
// a thermal mass, or a charging capacitor all behave.
static float g_pv = 0.0f;

static float plant_step(float u, float dt)
{
    g_pv += (u - g_pv) * (dt / PLANT_TAU);
    return g_pv;
}

// ----- task
static void control_task(void *arg)
{
    (void)arg;

    pid_controller_t ctl = {
        .dt      = DT_MS / 1000.0f,
        .out_min = OUT_MIN,
        .out_max = OUT_MAX,
        .kd      = 0.0f,          // PI, not PID -- see the walkthrough
#ifdef MODE == 0
        .kp = 1.5f, .ki = 0.0f, .anti_windup = true,
#elif MODE == 1
        .kp = 1.5f, .ki = 1.5f, .anti_windup = false,
#else
        .kp = 1.5f, .ki = 1.5f, .anti_windup = true,
#endif
    };

    const char *mode_name =
        (MODE == 0) ? "P-only" :
        (MODE == 1) ? "PI, NO anti-windup" : "PI, anti-windup";

    ESP_LOGW(TAG, "mode=%s kp=%.2f ki=%.2f out_max=%.0f",
        mode_name, ctl.kp, ctl.ki, (double)OUT_MAX);
    printf("t,setpoint,pv,u\n");          // CSV header for the plotter

    float peak = 0.0f;
    int n = (RUN_SECONDS * 1000) / DT_MS;
    TickType_t next = xTaskGetTickCount();

```

```

    for (int i = 0; i < n; i++) {
        float u = pid_step(&ctl, SETPOINT, g_pv);
        float pv = plant_step(u, ctl.dt);
        if (pv > peak) peak = pv;

        printf("%.2f,%.2f,%.2f,%.2f\n", i * ctl.dt, (double)SETPOINT,
            (double)pv, (double)u);

        // Absolute-deadline timing. The controller's dt is a CONSTANT in the
        // maths above, and it is only correct if the real interval matches.
        vTaskDelayUntil(&next, pdMS_TO_TICKS(DT_MS));
    }

    ESP_LOGW(TAG, "=====");
    ESP_LOGW(TAG, " mode      : %s", mode_name);
    ESP_LOGW(TAG, " final pv   : %.2f", (double)g_pv);
    ESP_LOGW(TAG, " offset    : %.2f", (double)(SETPOINT - g_pv));
    ESP_LOGW(TAG, " peak      : %.2f", (double)peak);
    ESP_LOGW(TAG, " overshoot : %.2f", (double)(peak > SETPOINT ? peak - SETPOINT : 0.0f));
    ESP_LOGW(TAG, "=====");
    vTaskDelete(NULL);
}

void app_main(void)
{
    xTaskCreate(control_task, "ctl", 4096, NULL, 5, NULL);
}

```

18.6 Code walkthrough, line by line

typedef struct { ... } pid_controller_t;

Members fall into two groups worth distinguishing. **Configuration** — gains, `dt`, limits, the anti-windup flag — set once and never touched by the algorithm. **State that must persist** — `integ`, `prev_meas`, `started` — which is why the function cannot be stateless.

Bundling state in a struct rather than file-scope globals means you can run several independent controllers — two motors, or a cascaded inner and outer loop — without interference. Same pattern as every driver handle in this series, applied to an algorithm.

float pid_step(pid_controller_t *p, float sp, float meas)

Takes the controller by **pointer**, because it must write back to `integ`, `prev_meas` and `started`. Pass by value and every update is lost on return, giving a permanently proportional-only response — a subtle bug that looks like bad tuning.

Note the shape: setpoint and measurement in, command out, **no hardware anywhere**. That makes it unit-testable exactly like the Lab 11 feature maths — feed it a known sequence, assert the output.

float deriv = p->started ? (meas - p->prev_meas) / p->dt : 0.0f;

Two decisions in one line. **Derivative on the measurement**, for the derivative-kick reason in the comment. And the **ternary guard**: on the first call `prev_meas` holds whatever the struct was initialised

with, and differentiating against it produces an enormous spurious spike. Returning zero for one cycle costs nothing and removes the problem entirely.

```
float integ_trial = p->integ + err * p->dt;
```

Multiplying by `dt` is what makes this a real integral rather than a running sum. It makes the accumulated value represent error-seconds, so changing the sample rate leaves `ki` meaning the same thing. A bare `integ += err` would double when you doubled the rate.

Computing it into a *trial* variable rather than committing immediately is what allows the anti-windup decision below.

```
out = kperr + kiinteg_trial - kd*deriv;
```

The minus on D is correct and follows from differentiating the measurement. If the measured value is rising, the derivative is positive, and you want to *reduce* drive to avoid overshooting. Get that sign wrong and damping becomes positive feedback — at least an obvious failure rather than a subtle one.

```
if (!p->anti_windup || err < 0.0f) p->integ = integ_trial;
```

The whole lab in one line. Read it as two cases.

Anti-windup off: the condition short-circuits true, the integral is always committed, and it inflates while the output is pinned. That is windup, and it is what MODE 1 demonstrates.

Anti-windup on: commit only when `err < 0`, meaning the error has reversed and integrating would now pull the output back *down* off the ceiling. While still climbing, the integral is frozen.

The lower-limit branch is the mirror image, and both are needed — a controller that only guards the ceiling winds up in the negative direction instead.

```
.kd = 0.0f
```

This is a **PI controller, not PID**, and that is a deliberate choice rather than a simplification.

Be precise about why, because the loose version of this claim is false. The *plant* is first order and has no momentum of its own — but the **closed loop**, once integral action is added, is second order and *can* overshoot. It demonstrably does: MODE 2 peaks at 102.46. So "a first-order plant cannot overshoot" is not true of the loop you are actually running.

The honest reason for `kd = 0` is that this plant has no measurement noise and no transport delay, so derivative action buys very little here while being the term most likely to amplify noise on real hardware. A great many industrial loops are PI for exactly that trade.

```
OUT_MAX 115.0f
```

Only slightly above the setpoint of 100, deliberately. It guarantees the output saturates during the climb, which is what makes windup demonstrable at all. Set it to 500 and the controller would never hit its limit and the anti-windup branch would never execute.

```
printf("%.2f,%.2f,%.2f,%.2f\n", ...)
```

CSV, not `ESP_LOGI`, deliberately. The logging macro prefixes a timestamp, level and tag, which is noise you would then strip. Raw CSV pastes straight into a serial plotter or a spreadsheet. **Plotting is how you tune**; staring at numbers is not.

The `(double)` casts are required: `printf` is variadic, so `float` arguments are promoted to `double` anyway, and casting explicitly documents that and silences `-Wdouble-promotion`.

```
vTaskDelayUntil(&next, pdMS_TO_TICKS(DT_MS));
```

Absolute-deadline timing: sleep until `next`, then advance `next` by exactly one period, so wake-ups land on a fixed grid regardless of how long the computation and the `printf` took.

This is not tidiness. `dt` is a *constant* in the arithmetic above, and it is only correct if the real interval matches it. A plain `vTaskDelay` would make the true period (work + 10 ms), varying with print length, and the maths would be quietly wrong in a way that looks like bad tuning. **If a control loop ever behaves differently when you add logging, this is why.**

18.7 Phase 2 — build, flash, run

```
idf.py build
idf.py -p COM6 flash monitor
```

Run all three modes. Change `MODE`, rebuild, reflash.

KEY POINT

Expected summary lines

```
MODE 0  mode : P-only           final pv : 60.00  offset : 40.00  overshoot : 0.00
MODE 1  mode : PI, NO anti-windup final pv : 100.00 offset : 0.00  overshoot : 14.58
MODE 2  mode : PI, anti-windup   final pv : 100.00 offset : 0.00  overshoot : 2.46
```

Those three numbers — **40.00, 14.58, 2.46** — are the ones to memorise. They are the answer to "have you actually tuned a control loop?"

18.8 Phase 3 — prove it

#	Do this	Expected
1	Run MODE 0.	Settles at 60.00 , a permanent 40-unit offset. Stable and permanently wrong.
2	Run MODE 1.	Reaches 100 but peaks at 114.58 and takes about 15 s to come back.
3	Run MODE 2.	Peaks at 102.46 . Overshoot cut by 83% with no change to the gains.
4	Capture the CSV from MODE 1 and MODE 2 and plot both.	Two curves that separate visibly. This plot is portfolio material — save it.
5	Set <code>OUT_MAX</code> to 200 and rerun MODE 1 and MODE 2.	Both now peak at 114.51 — identical. Removing saturation removes <i>windup</i> but leaves ordinary PI overshoot untouched, and anti-windup then has nothing left to do. See §18.9.2.

WARNING**Case 5 corrects a claim an earlier edition of this text got wrong**

That edition said raising `OUT_MAX` would make overshoot "collapse to near zero." **It does not.** Re-running the model gives **14.51** at `OUT_MAX` 500, and the same 14.51 at 5000. The claim was never measured before it was printed.

What is actually true is more useful: overshoot here has **two independent sources**, and only one is windup.

Ordinary PI transient. Integral action stores energy while the error is positive and keeps pushing after the setpoint is reached. This happens with no saturation anywhere, and is worth **14.51** units on this plant.

Windup. Extra accumulation that occurs only *while the output is pinned*. Its size depends on how long you spend saturated — which §18.9.2 measures directly.

18.9 Extensions, worked

18.9.1 — Tuning, with real numbers

WARNING**A fabricated claim, removed**

An earlier edition said "raise `kp` until the output oscillates — on this plant that is around `kp = 12`."

That number was invented and the behaviour is not achievable. A first-order lag with no transport delay *cannot be driven unstable by proportional action alone*; there is no finite ultimate gain. Simulated: at `kp = 12` the output settles smoothly at 92.31 with zero ripple, and even at `kp = 1000` the ripple is 0.78 and is a discretisation artefact rather than a real oscillation.

So Ziegler-Nichols step one — "increase gain until sustained oscillation" — **has no answer on this plant.** Recognising when a standard method does not apply is worth more than pretending it did.

Tune against the measured step response instead. Set `MODE 2`, change the two gains, rebuild, and read the summary block.

KEY POINT

Results[SIMULATED] — from the exact model in §18.5, anti-windup on, `OUT_MAX` 115

kp	ki	Peak	Overshoot	Settle to 2%	Comment
1.5	1.5	102.46	2.46	8.88 s	the lab default — conservative
3.0	2.0	100.37	0.37	6.14 s	better on both axes — faster AND less overshoot
6.0	2.0	100.00	0.00	9.91 s	no overshoot at all, but <i>slower</i> to settle

The third row is the one worth understanding. More proportional gain is **not** monotonically better: past a point the output spends longer pinned at `OUT_MAX`, so the loop takes longer to settle even though it never overshoots. That is a real trade you can only see by measuring.

[MEASURE] Run $k_p 3.0 / k_i 2.0$ on your board and record the printed peak and offset. If your figures differ materially from 100.37, note it — that is a genuine observation about tick granularity and float behaviour on target, not a failure.

18.9.2 — How much of the overshoot is actually windup?

Case 5 claims windup and ordinary PI overshoot are separate effects. Here is the sweep that proves it, with the script, so the table is reproducible rather than asserted.

```
# windup_sweep.py -- runs on your PC in about a second. No board needed.
DT, SP, TAU, N = 0.01, 100.0, 3.0, 10000
```

```
def run(kp, ki, omax, aw):
    pv = integ = 0.0
    peak = 0.0
    sat = 0
    for _ in range(N):
        err = SP - pv
        trial = integ + err * DT
        out = kp * err + ki * trial
        if out > omax:
            out = omax; sat += 1
            if (not aw) or err < 0: integ = trial
        elif out < 0.0:
            out = 0.0; sat += 1
            if (not aw) or err > 0: integ = trial
        else:
            integ = trial
        pv += (out - pv) * (DT / TAU)
        peak = max(peak, pv)
    return max(0.0, peak - SP), sat * DT

print("OUT_MAX  sat_time  no-AW  with-AW  windup")
for omax in (102, 105, 110, 115, 130, 200, 500):
    a, sa = run(1.5, 1.5, omax, aw=False)
    b, _ = run(1.5, 1.5, omax, aw=True)
    print(f"{omax:7d} {sa:8.2f}s {a:7.2f} {b:8.2f} {a-b:8.2f}")
```

KEY POINT**Output[SIMULATED]******

OUT_MAX	Time saturated	Overshoot, no AW	Overshoot, with AW	Windup's share
102	100.00 s	2.00	0.33	1.67
105	47.99 s	5.00	0.82	4.18
110	24.66 s	9.97	1.64	8.33
115	16.83 s	14.58	2.46	12.12
130	8.62 s	22.92	4.91	18.02
200	0.00 s	14.51	14.51	0.00
500	0.00 s	14.51	14.51	0.00

Three findings, all worth being able to state:

With no saturation, anti-windup does nothing. The last two rows are identical across both columns, and the 14.51 there is the pure PI transient. That is the correct version of the claim case 5 used to make.

Windup's contribution is not monotonic in the limit. It peaks near `OUT_MAX` 130 and falls again at 102, because a very tight limit stops the output overshooting much at all regardless of the integrator.

At the lab's own setting the split is about 2.5 ordinary and 12 windup. So most of MODE 1's 14.58 genuinely is windup, which is why the lab is built at `OUT_MAX` 115.

18.9.3 — Seeing derivative kick

Prove the derivative-on-measurement decision matters, in one edit. Change the derivative line to differentiate the *error* instead:

Here is the **complete** replacement for the derivative block in `pid_step()` — three lines out, five in, plus one sign change on the output line.

```
// ---- DELETE these three lines:----
// float deriv = p->started ? (meas - p->prev_meas) / p->dt : 0.0f;
// p->prev_meas = meas;
// p->started = true;

// ---- PUT THIS IN THEIR PLACE (deliberately wrong, to see the kick): ----
static float prev_err = 0.0f;
float deriv = p->started ? (err - prev_err) / p->dt : 0.0f;
prev_err = err;
p->prev_meas = meas;
p->started = true;

// ---- then change the output line's D sign from minus to PLUS: ----
float out = p->kp * err + p->ki * integ_trial + p->kd * deriv;
```

The sign flips because differentiating the error instead of the measurement reverses the term's polarity.

Set `kd = 0.5f` and add a setpoint step at $t = 20$ s:

```
float sp = (i * ctl.dt < 20.0f) ? 100.0f : 50.0f;
float u  = pid_step(&ctl, sp, g_pv);
```

KEY POINT

Result

At the instant of the setpoint change the error jumps by 50, so its derivative is momentarily **5000** (50 divided by `dt` of 0.01), and the output slams to its limit for one sample. In the CSV you will see a single-sample spike in the `u` column.

With derivative on the measurement, that spike is **absent** — because a physical process value cannot jump instantly. In a real actuator the spike is an audible bang: a motor lurching, a valve slamming. This is a small change that marks out someone who has run a loop rather than only read the equation.

18.10 Docs and references

Topic	Where
<code>vTaskDelayUntil</code> and periodic tasks	FreeRTOS API reference
Anti-windup schemes	Astrom & Hagglund, <i>PID Controllers</i> , ch. 3 — the standard reference
Ziegler-Nichols tuning	Any controls text; the original 1942 paper is short and readable
Serial plotting	Arduino IDE Serial Plotter, or paste the CSV into a spreadsheet

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Name collision. `pid_t` is a POSIX type declared in `<sys/types.h>` for process identifiers. Rename the struct to `pid_controller_t`. Also add the missing `<math.h>`, `<stdbool.h>` and FreeRTOS includes.

Anti-windup refinement. The listing backs the integrator out whenever the output saturates, in either direction. That prevents windup but also blocks integration that would bring the controller out of saturation. The standard refinement is **conditional integration**: inhibit the update only when the error would drive the output further into the limit it is already against. See §26.7 for the corrected function.

The corrected PID step function

Two changes from the §18.5 listing: the type is renamed, and the anti-windup becomes conditional so the integrator can still recover the controller out of saturation.

```
#include <math.h>
#include <stdbool.h>

typedef struct { float kp, ki, kd, dt, out_min, out_max;
                float integ, prev_meas; bool started; } pid_controller_t; // was pid_t -- POSIX
collision

float pid_step(pid_controller_t *p, float sp, float meas) {
    float err = sp - meas;
    float deriv = p->started ? (meas - p->prev_meas) / p->dt : 0.0f;
    p->prev_meas = meas; p->started = true;

    float integ_trial = p->integ + err * p->dt; // try the update, do not commit yet
    float out = p->kp*err + p->ki*integ_trial - p->kd*deriv;

    if (out > p->out_max) {
        out = p->out_max;
        // commit ONLY if this error pulls us back down out of the ceiling
        if (err < 0.0f) p->integ = integ_trial;
    } else if (out < p->out_min) {
        out = p->out_min;
        if (err > 0.0f) p->integ = integ_trial; // pulls us back up off the floor
    } else {
        p->integ = integ_trial; // linear region: always integrate
    }
    return out;
}
```

KEY POINT**Why this is better than backing the step out**

The §18.4 version subtracts the step whenever the output is saturated. Consider overshooting: the output is pinned at `out_max` and the error has just gone *negative*. The integrator now wants to shrink, which is exactly what you want — but the original code undoes that shrink, so the controller stays wound up longer than necessary. The conditional form blocks only the integration that would push further into the limit.

The measured numbers in §18.7 (offset 40.00, overshoot 14.58 without anti-windup, 2.46 with) were produced by the §18.4 version and remain valid for it. Re-run them with this version and record both; the comparison is itself a good exercise.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 18.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18 ←	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.